课程实验报告

一、手写数字识别

1. 实验目标与环境

这里使用 MNIST 数据集训练 LeNet-5，完成 0～9 共 10 类手写数字分类。输入图像是灰度图，原始的 28×28 像素图像先缩放到 32×32，再做归一化。训练和评测均使用 MindSpore。

官方训练集有 60,000 张图像。我用固定随机种子 42 将它分成 54,000 张训练图像和 6,000 张验证图像。验证集只用于每轮评估和选权重，不参与梯度更新；官方测试集的 10,000 张图像留到模型确定后再使用。

2. 模型与训练设置

LeNet-5 包含两组卷积和最大池化层，以及三层全连接层。卷积通道数为 1→6→16，全连接层维度为 400→120→84→10。前两层卷积和前两层全连接后接 ReLU，输出层用 Softmax 交叉熵损失完成分类。

训练使用 Momentum 优化器，学习率为 0.01，动量为 0.9，batch size 为 32，共训练 10 轮。每轮结束后计算验证集 loss 和准确率，优先保留验证准确率较高的权重；准确率相同时，再比较验证 loss。脚本会保存每轮指标、模型权重和训练曲线。

3. 训练曲线与测试结果

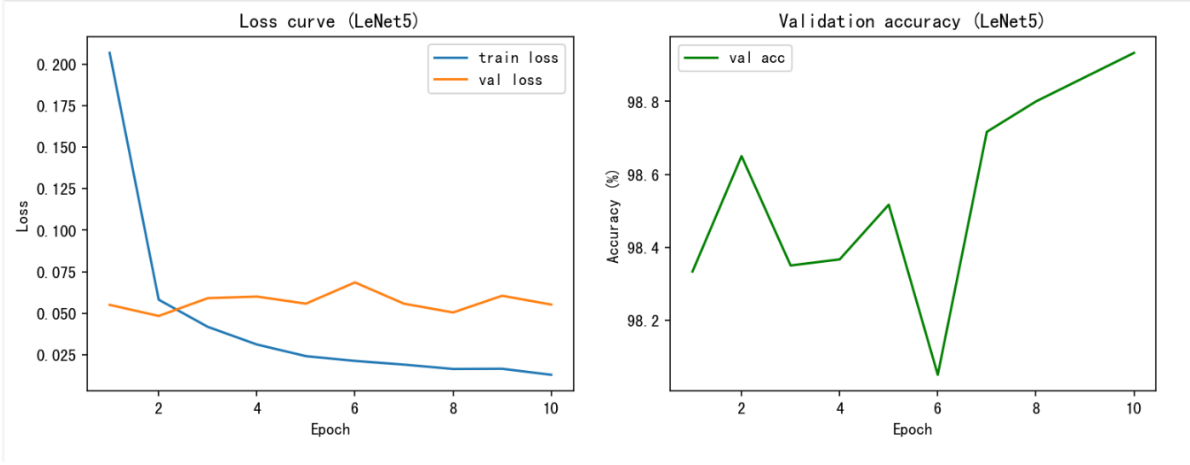


图 1　LeNet-5 的训练曲线。训练 loss 从第 1 轮约 0.207 降到第 10 轮约 0.012。验证 loss 在 0.049～0.069 之间波动，并没有跟着训练 loss 一直下降。验证准确率整体保持在 98% 以上，第 10 轮最高，约为 98.93%。

最后用第 10 轮权重 lenet-10.ckpt 评测官方测试集。测试集共 10,000 张图像，batch size 设为 100，共 100 批，准确率为 98.94%，平均 loss 为 0.0439。这个结果来自独立测试集，和曲线中的验证准确率不是同一个指标。

4. 手写识别结果小结

LeNet-5 在验证集和官方测试集上的表现都比较稳定。后几轮训练 loss 继续下降，验证 loss 却有波动，所以选模型时还要看验证集表现。最终测试集准确率为 98.94%。

5. 对应的实验文件

训练文件：zju/手写和垃圾分类/lenet_mnist.py at master · Yangguangxiaokeji/zju

测试文件：zju/手写和垃圾分类/lenet_mnist_main.py at master · Yangguangxiaokeji/zju

二、垃圾分类

1. 实验目的

垃圾分类实验在浙江大学 momodel 平台（mo.zju.edu.cn）完成。任务是基于 MindSpore 和 MobileNetV2 预训练模型，对 26 类垃圾图像进行分类，并提交模型参加平台系统测试。

2. 实验环境

项目	说明
训练方式	momodel 平台离线作业，训练过程在平台完成
平台运行环境	Python 3.10.19；MindSpore 2.11.0、NumPy 1.26.4、OpenCV 4.11.0、Matplotlib 3.10.8、EasyDict 1.13
测试方式	平台系统测试，使用未参与训练的测试图像
深度学习框架	MindSpore，GRAPH_MODE
模型结构	MobileNetV2 backbone + 分类 head（GlobalPooling + Dense），分阶段选择是否解冻 backbone
预训练权重	mobilenetv2-200_1067_cpu_gpu.ckpt（平台提供，ImageNet 预训练）
数据集	garbage_26x100，26 类垃圾，每类约 100 张
输入尺寸	224 x 224，batch_size = 24

26 个类别包括塑料瓶、帽子、报纸、易拉罐、玻璃器皿、纸板、电池、荧光灯、橘子皮、菜叶、蛋壳、香蕉皮等，对应平台固定的标签字典 inverted（0~25）。

数据集中共有 2343 张训练图像、130 张验证图像，验证集每类 5 张。训练按 batch_size=24 分批，每轮有 97 个完整批次，共使用 2328 张图像，其余 15 张因 drop_remainder=True 被舍弃。验证按 batch_size=10 分批，13 个批次可以覆盖全部验证图像。

3. 总体方案

训练分成两个阶段：

- 阶段一（train_main.py）：冻结预训练 backbone，只训练分类 head。
- 阶段二（train_main_all.py）：加载阶段一最优权重，解冻 backbone 和 head，直接读取图片进行微调。先采用统一学习率，再尝试分组学习率。

阶段一按完整批次提取训练图像的特征，将 feature map 和标签保存为 feature_i.npy、label_i.npy，放在 results/garbage_26x100_features 下。之后每轮直接读取缓存，只对分类 head 做前向和反向计算。由于 backbone 已冻结，不需要重复计算特征，训练时间也会短一些。

缓存也带来一个限制：随机裁剪、翻转和颜色调整只在生成缓存时执行一次，后面的 head 训练重复使用同一批特征。全网微调则每轮读取原图，重新做随机增强，同时更新特征提取网络。

4. 实验过程

4.1 原始文件的调整

原始流程没有逐轮验证，提交文件中的模型定义也没有完全对齐训练代码。我对 train_main.py、main.ipynb 和 main.py 做了以下调整：

1. 每轮用 val 集计算 loss 和准确率，按验证准确率选权重。
2. 每次训练使用独立的时间戳目录保存 checkpoint 和 training_curve.png，并把超参数、目录和曲线写入 results/训练轨迹.md。
3. 提取特征时复用已有缓存，避免重复处理。
4. main.py 只保留模型加载和 predict()，训练代码与提交代码统一使用 has_bias=True 的 head，并核对权重路径、归一化方式和通道顺序。

最初提交得分为 55。另一次提交因为仍包含训练流程、Head 不匹配以及权重路径错误，得分降到 45。修正后再做基准训练，结果见下一节。

4.3 改造后的基准训练与结果

文件调整完成后，先冻结 MobileNetV2 backbone，只训练分类 head，得到一组基准结果。后面的 head 参数实验以它作对照；全网微调则从 head 第 2 组的最佳权重开始。

超参数	取值
epochs	10
lr_max	0.01
decay_type	constant
momentum	0.8
weight_decay	0.0001
batch_size	24
优化器	nn.Momentum
损失函数	SoftmaxCrossEntropyWithLogits(sparse=True, mean)

验证准确率在第 9 轮最高，为 91.54%，对应权重 mobilenetv2-9.ckpt，系统测试得分也是 91.54。全网微调前，我在同一验证集上重新评估了这个权重，准确率为 0.9153846，验证 loss 为 0.337953。

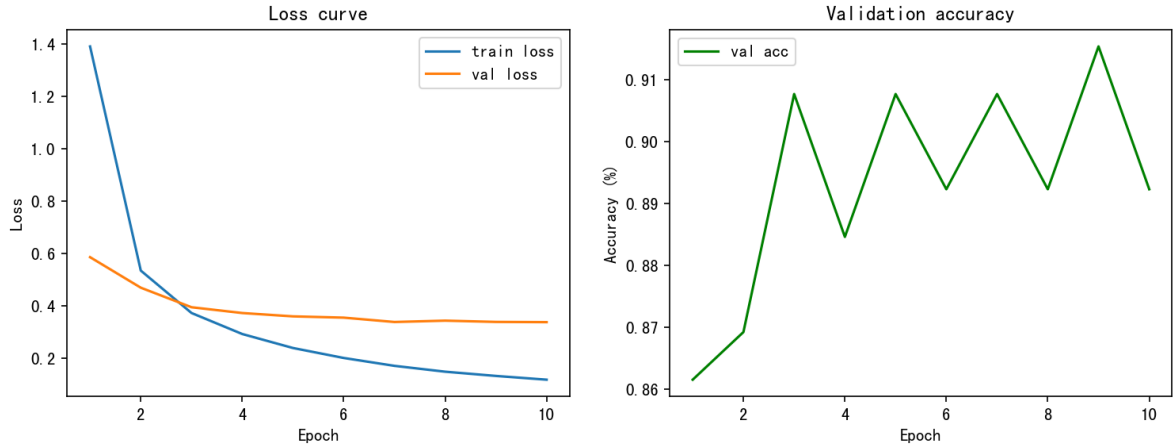


图 1　阶段一训练曲线。第 10 轮训练 loss 还在下降，但验证准确率低于第 9 轮，所以最终使用第 9 轮权重。右图纵轴按 0～1 显示，0.9154 就是约 91.54%。

4.4 超参数调整实验与结果对比

4.4.1 三组单因素实验

我在平台上完成了第 1、2、3 组三次 head 训练。三组都冻结 backbone，读取同一份特征缓存，只改变学习率或衰减方式。共同配置为 epochs=10、momentum=0.8、weight_decay=0.0001、batch_size=24、eval_batch_size=10、reduction=mean，输入尺寸为 224×224；数据路径、缓存路径和模型结构没有变化。其他推荐组未参加这次比较。

训练参数、最佳轮次和验证指标来自平台训练轨迹截图，系统测试分数来自三组实际提交结果。表中的验证 loss 均对应选中的轮次，原基线的 0.337953 来自第 4.3 节的复评。

实验	lr_max	decay_type	最佳轮次	最佳验证准确率（%）	对应验证 loss	系统测试得分	较原基线得分变化
原基线	0.01	constant	9	91.54	0.337953	91.54	—
第 1 组：降低学习率	0.005	constant	7	90.77	0.366998	90.00	-1.54
第 2 组：提高学习率	0.02	constant	5	90.77	0.306713	**93.85**	**+2.31**
第 3 组：余弦衰减	0.01	cosine	6	90.77	0.344853	91.54	0.00

三组的最高验证准确率原始记录都是 0.907692，约为 90.77%。验证集有 130 张图像，正确识别 118 张，比原基线少 1 张。三组验证准确率相同，系统测试分数却不同，说明两者不能直接画等号。

各组参数、checkpoint 目录和实际测试权重如下。每个目录中都保留了对应的 training_curve.png。

实验	checkpoint 文件夹	系统测试所用权重
第 1 组	`results/ckpt_mobilenetv2-20261109-153144/`	`mobilenetv2-7.ckpt`
第 2 组	`results/ckpt_mobilenetv2-20261109-153207/`	`mobilenetv2-5.ckpt`
第 3 组	`results/ckpt_mobilenetv2-20261109-153237/`	`mobilenetv2-6.ckpt`

4.4.2 训练曲线与结果分析

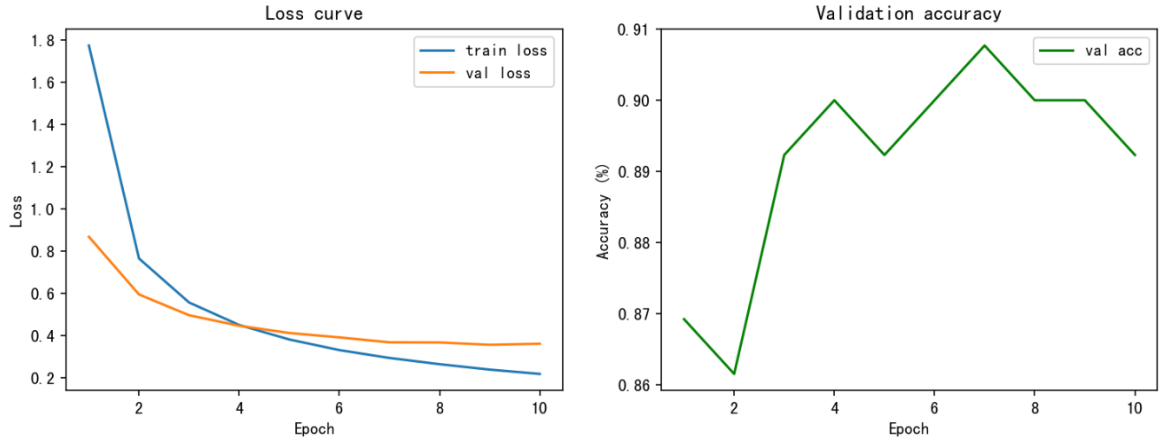


图 2　第 1 组训练曲线，截取自平台训练轨迹。训练 loss 从约 1.78 降到 0.22，验证 loss 前几轮下降较快，后面趋于平缓。第 7 轮验证准确率最高，为 90.77%，验证 loss 为 0.366998；之后准确率回落。该轮权重的系统测试得分为 90.00，比基线低 1.54 分，降低学习率没有带来改善。

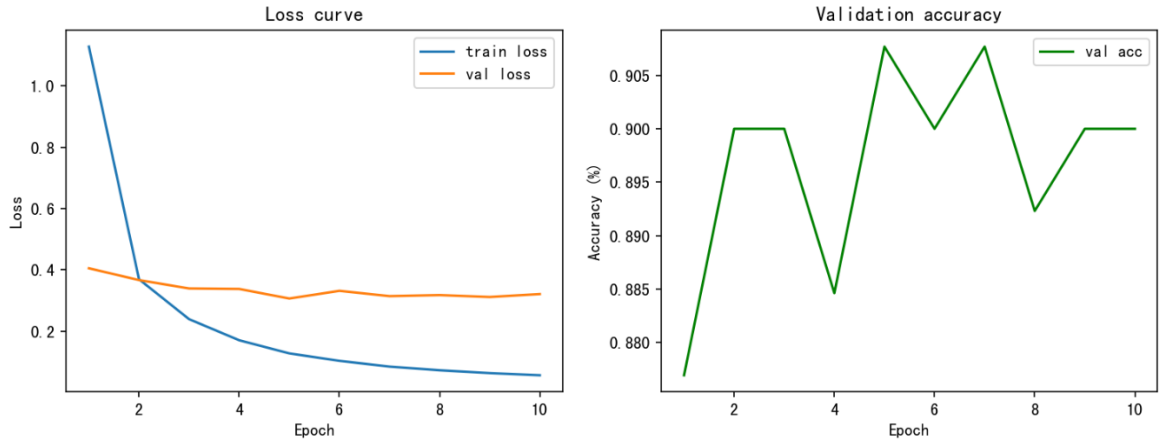


图 3　第 2 组训练曲线，截取自平台训练轨迹。训练 loss 从约 1.12 降到 0.06，速度比第 1 组快；验证 loss 后几轮在 0.31 ~ 0.34 之间波动，没有继续跟着训练 loss 下降。第 5 轮首次达到最高验证准确率 90.77%，验证 loss 为 0.306713，按脚本规则选用 mobilenetv2-5.ckpt。该权重的系统测试得分为 93.85，比基线高 2.31 分，也是目前记录中最高的系统测试分数。

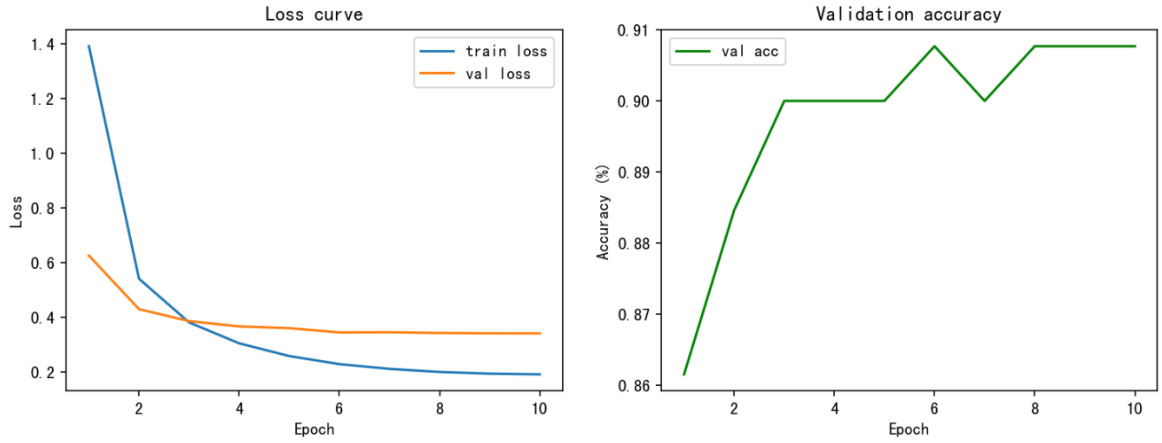


图 4　第 3 组训练曲线，截取自平台训练轨迹。训练 loss 从约 1.39 降到 0.19，验证 loss 前期下降，后期基本稳定。第 6 轮首次达到最高验证准确率 90.77%，验证 loss 为 0.344853，之后几轮准确率保持不变。mobilenetv2-6.ckpt 的系统测试得分为 91.54，与基线相同，余弦衰减在这组实验中没有带来提升。

在冻结 backbone、只训练 head 的条件下，这三组里把学习率从 0.01 提高到 0.02 的结果最好。

4.5 全网微调：统一学习率实验

这次统一学习率实验重新开始，之前的记录不再使用。训练从第 2 组 head-only 权重开始，解冻 backbone 和 head，按原流程读取图像并在每轮重新做随机增强。

参数	取值
epochs	8
backbone_lr_max / head_lr_max	0.002 / 0.002
decay_type / warmup_steps	cosine / 0
momentum / weight_decay	0.9 / 0.0001
训练 / 验证 batch_size	24 / 10

训练脚本是 train_main_all_uniform_lr.py。checkpoint 和训练曲线保存到 results/ckpt_mobilenetv2_all_uniform_lr-<时间戳>/，训练轨迹追加写入 results/训练轨迹.md。起始权重为第 2 组的最佳权重 ./results/ckpt_mobilenetv2-20261109-153207/mobilenetv2-5.ckpt。

平台曲线中，epoch 0 表示起始权重的基线。训练 loss 从第 1 轮开始下降，验证准确率在第 4 轮首次达到最高值，约为 93.85%，后面有保持也有回落，所以把第 4 轮记为最佳轮次。该轮权重的系统测试得分为 **86.15**，低于起始权重对应的 93.85，也低于阶段一基线的 91.54。

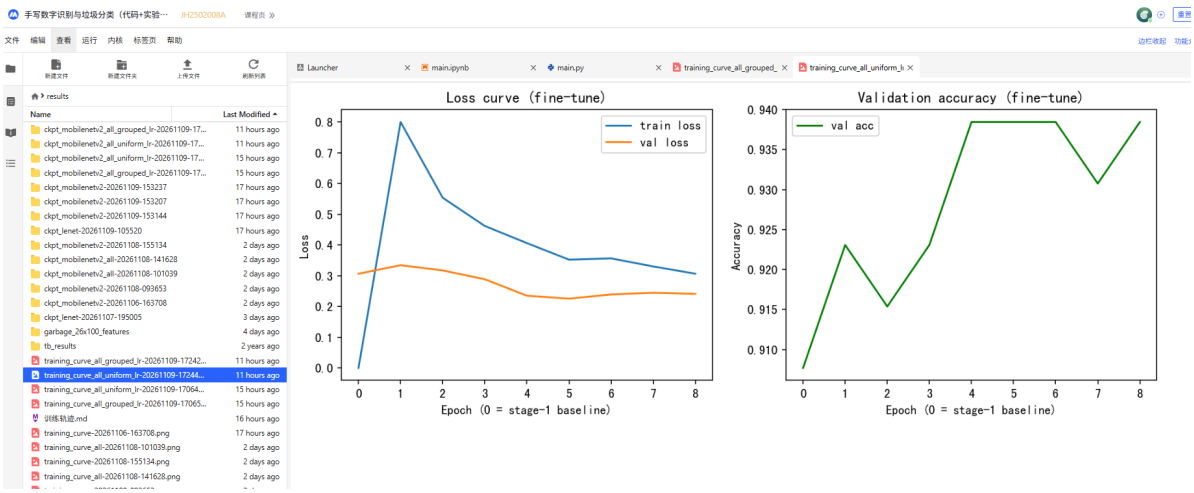


图 5　全网统一学习率训练曲线。验证集上的准确率高于起始权重，但系统测试分数下降了。

4.6 全网微调：分组学习率实验

分组学习率实验也重新进行，之前的日志和曲线不计入这里。它与统一学习率组使用相同的模型、起始权重、轮数和训练流程。head-only 第 2 组的 lr=0.02 虽然表现较好，但当时 backbone 是冻结的，调度方式也是 constant，不能直接搬到全网微调中。本组保持 head_lr_max=0.002，只把 backbone_lr_max 降到 0.0001，用来观察 backbone 学习率的影响。

参数	取值
epochs	8
backbone_lr_max / head_lr_max	0.0001 / 0.002
decay_type / warmup_steps	cosine / 0
momentum / weight_decay	0.9 / 0.0001
训练 / 验证 batch_size	24 / 10

训练脚本是 train_main_all_grouped_lr.py。checkpoint 和训练曲线保存到 results/ckpt_mobilenetv2_all_grouped_lr-<时间戳>/，训练轨迹同样追加写入 results/训练轨迹.md。起始权重仍为 ./results/ckpt_mobilenetv2-20261109-153207/mobilenetv2-5.ckpt。

平台曲线显示，验证准确率最高约为 91.54%，第 3 轮达到峰值，后面有波动。按首次达到最高准确率的规则，最佳轮次是第 3 轮。该轮权重的系统测试得分为 **91.54**，与阶段一基线相同，低于 head-only 第 2 组的 93.85。

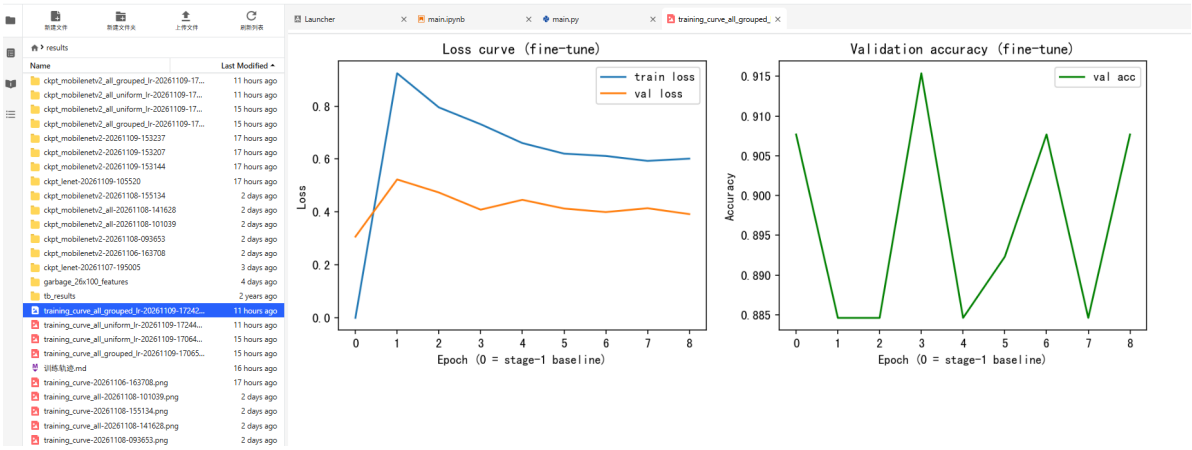


图 6　全网分组学习率训练曲线。降低 backbone 学习率后，系统测试得分回到阶段一基线。

4.7 对结果差异的分析

两组全网微调都从 head-only 第 2 组权重开始。统一学习率组的验证准确率最高，但系统测试只有 86.15；分组学习率组的验证准确率约 91.54，系统测试也是 91.54。前者在验证集上的提升没有出现在系统测试中，说明全网微调不能只看验证准确率选权重。分组学习率减小了 backbone 的更新幅度，结果没有超过 head-only 第 2 组，但回到了阶段一基线。

5. 实验结果汇总

实验	超参数配置	最佳轮次	验证准确率（%）	系统测试得分
初始提交	notebook 原始代码	—	—	55
对齐前的提交	main.py 含训练流程、Head 不匹配	—	—	45
阶段一基线	head 训练；10 轮，lr=0.01，constant，momentum=0.8	9	91.54	91.54
head 第 1 组，降低学习率	10 轮，lr=0.005，constant，momentum=0.8	7	90.77	90.00
head 第 2 组，提高学习率	10 轮，lr=0.02，constant，momentum=0.8	5	90.77	**93.85**
head 第 3 组，余弦衰减	10 轮，lr=0.01，cosine，momentum=0.8	6	90.77	91.54
全网统一学习率	8 轮，backbone/head lr_max=0.002，cosine	4	93.85	**86.15**
全网分组学习率	8 轮，backbone=0.0001、head=0.002，cosine	3	91.54	91.54

现有记录中最高的系统测试得分是 93.85，来自 head 第 2 组（lr=0.02、constant）的第 5 轮权重：results/ckpt_mobilenetv2-20261109-153207/mobilenetv2-5.ckpt。原基线和全网分组学习率都是 91.54，全网统一学习率为 86.15。全网微调没有超过单独调整 head 学习率的第 2 组。

6. 实验中遇到的问题

1. 模型定义必须和 checkpoint 对得上。参数不匹配时，任务未必直接报错，需要检查加载警告和未加载参数，尤其是 Dense 层的 bias。
2. 平台测试入口由 notebook 勾选的 cell 生成。训练脚本改过以后，要重新核对生成的 main.py，避免测试时又执行训练。
3. checkpoint、日志和曲线都按时间戳分目录保存，这样可以把每次训练的准确率、曲线和测试权重对应起来。

7. 实验心得

最初得分从 55 降到 45 时，我先怀疑训练参数，后来才发现提交文件和模型结构没有对齐。改好之后分数回到 91.54。遇到这种异常，先核对测试入口、模型结构和权重，通常比直接改参数更快。

全网微调把这个差异表现得很明显。统一学习率组的验证准确率最高约 93.85%，系统测试却只有 86.15；分组学习率组的验证和系统测试都是 91.54。后面选模型时，我会同时看验证曲线和系统测试结果，不把验证集最高准确率直接当成最终成绩。

三组 head 单因素实验里，只有把学习率提高到 0.02 的第 2 组超过了基线，系统测试得到 93.85；降低学习率是 90.00，余弦衰减是 91.54。三组的最高验证准确率却完全一样，这也是为什么只看验证准确率会漏掉实际差异。

目前保留的权重是 head 第 2 组的 mobilenetv2-5.ckpt。全网统一学习率实验降到 86.15，分组学习率实验回到 91.54，都没有超过 93.85。epoch 0 候选机制只能保证选出的权重在当前验证集上不低于阶段一基线，不能保证系统测试分数也一起提高。

8.对应的实验文件